

BERT(Bidirectional Encoder Representations from Transformers)

트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델
 양방향 인코더- 입력 시퀀스를 양쪽 방향에서 처리해서 이전과 이후 단어를 모두 참조-> 단어의 의미%문맥 파악
 (기존 모델은 좌-> 우 방향으로 문맥 파악시 이전 단어만 고려)
 - 전이 학습에 주로 사용
 - 다른 작업에서 재사용 -> 적은 양의 데이터로도 높은 정확도 달성 가능
 - 모델 학습 시간 단축 가능
 - 학습 데이터의 다양성을 고려한 높은 성능

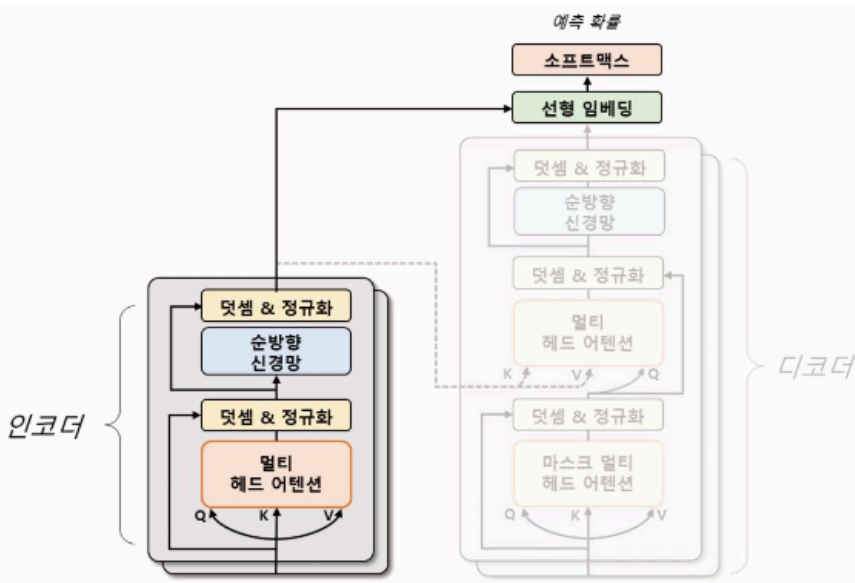


그림 7.13 트랜스포머와 BERT 모델 비교

트랜스포머 인코더 모듈만을 사용(양방향 처리)해 입력 문장 처리: 각 단어 임베딩-> 의미 벡터화->순차처리->문장 의미 추출

사전 학습 방법:

- **마스크된 언어 모델링(Masked Language Modeling, MLM)**: 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식

ex) I'm learning PyTorch -> I'm [] PyTorch : [] 에 들어가 단어 예측. => 문장 전체 의미 이해 능력 향상

- **다음 문장 예측(Next Sentence Prediction, NSP)**: 2개의 문장이 주어졌을 때, 2번째 문장이 첫번째 문장 다음에 오는 문장인지 여부 판단 작업. 2 문장이 연속적인 관계인지 예측

입력 문장에 특수 토큰들을 통해 모델이 학습하고 추론하는 과정에서 필요한 정보 제공

[CLS]토큰: 입력 문장의 시작 부분에 추가되는 토큰. 문장 분류 작업을 위한 정보 제공-> 입력 문장이 어떤 유형의 문장인지 미리 파악 가능(긍/부정, 객체 언급 등)

[SEP]토큰: 입력 문장 내에서 2개 이상의 문장을 구분하기 위해 사용되는 토큰. 입력 문장들을 독립적인 문장으로 인식, 각각의 문장에 대한 정보 파악 가능

[MASK] 토큰: 입력 문장 내에서 임의로 선택된 단어를 가리키는 특별한 토큰. 주어진 문장에서 일부 단어 가린 후 모델 학습&예측에 사용



그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

BERT 모델의 입력 임베딩과 손실 함수

[CLS] BERT는 트랜스포머 인코더를 포함한다 [SEP] 같은 형태로 변환
 텍스트 토큰 중 15%에 해당하는 단어를 대상으로 마스킹 수행-> 이 중 80%를 [MASK]토큰으로 대체, 10%는 어
 휘 사전에 존재하는 단어로 무작위 변경, 10%는 실제 토큰 그대로 사용

BART(Bidirectional Auto-Regressive Transformer)

BERT인코더와 GPT디코더를 결합한 Sequence-to-Sequence(Seq2Seq)구조

노이즈 제거 오토인코더(Denoising Autoencoder)로 사전 학습 : 입력 데이터에 잡음 추가 -> 원본 데이터 복원하
 도록 학습 하는 방식

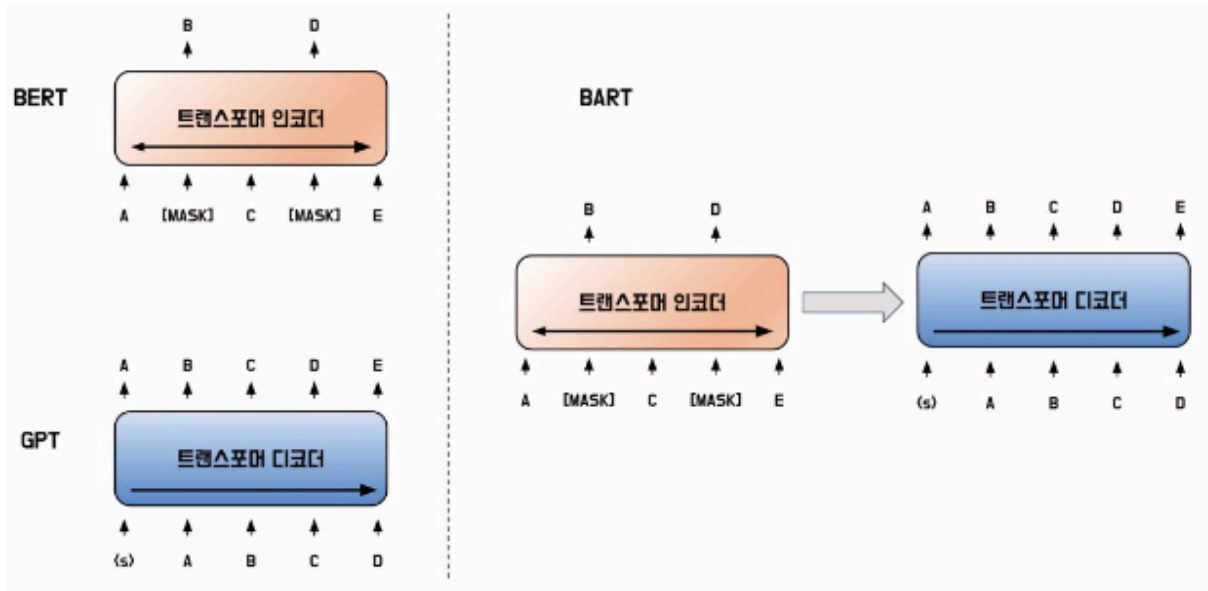


그림 7.15 BART 사전 학습 시각화

BERT가 **인코더** 학습 시 텍스트에 마스킹을 추가해 노이즈를 주는 방법 + GPT가 **디코더** 학습 시 다음에 등장할 단
 어를 예측하는 방법 => 문장 구조와 의미를 보존하면서 다양한 변형 학습 가능. 더 풍부한 언어적 지식 습득

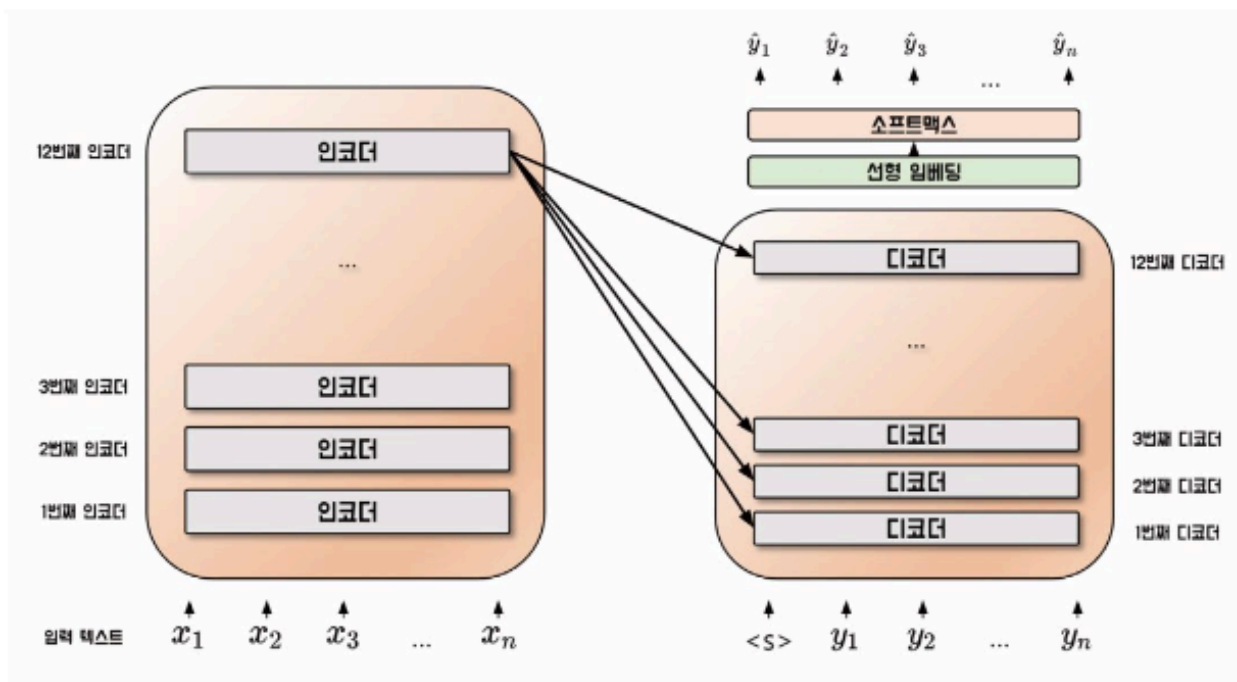


그림 7.16 BART 모델 구조

인코더+ 디코더-> 트랜스포머와 유사한 구조

트랜스포머: 모든 계층 사이의 어텐션 연산. => BART: 인코더의 마지막 계층과 디코더의 각 계층 사이 어텐션 연산

-> 정보 전달 최적화, 메모리 사용량 줄일 수 있음

사전 학습 방법:

BERT의 MLM: 입력 문장의 일부 토큰을 마스크 토큰으로 치환

- 중요한 단어 파악
- 문맥 이해 및 정보 추출능력 향상

다양한 노이즈 기법:

- 토큰 마스킹 기법
- 토큰 삭제 : 토큰 치환X, 삭제O. -> 불필요한 정보, 중요하지 않은 정보 자동 필터링
- 시간 단축, 일반화능력 향상

- **문장 순열:** 마침표를 기준으로 문장을 나눈 뒤 문장 순서 섞는 방법 -> 문장 내에서 단어 연결관계 파악
- **문서 회전:** 임의의 토큰으로 문서가 시작하도록 해서 문서의 원래 시작 토큰 맞춰서 문서 시작점 인식 능력 향상
- **텍스트 채우기:** 몇 개의 토큰을 하나의 구간으로 묶어서 마스크 토큰으로 대체 -> 누락된 단어 예측 유도

등..

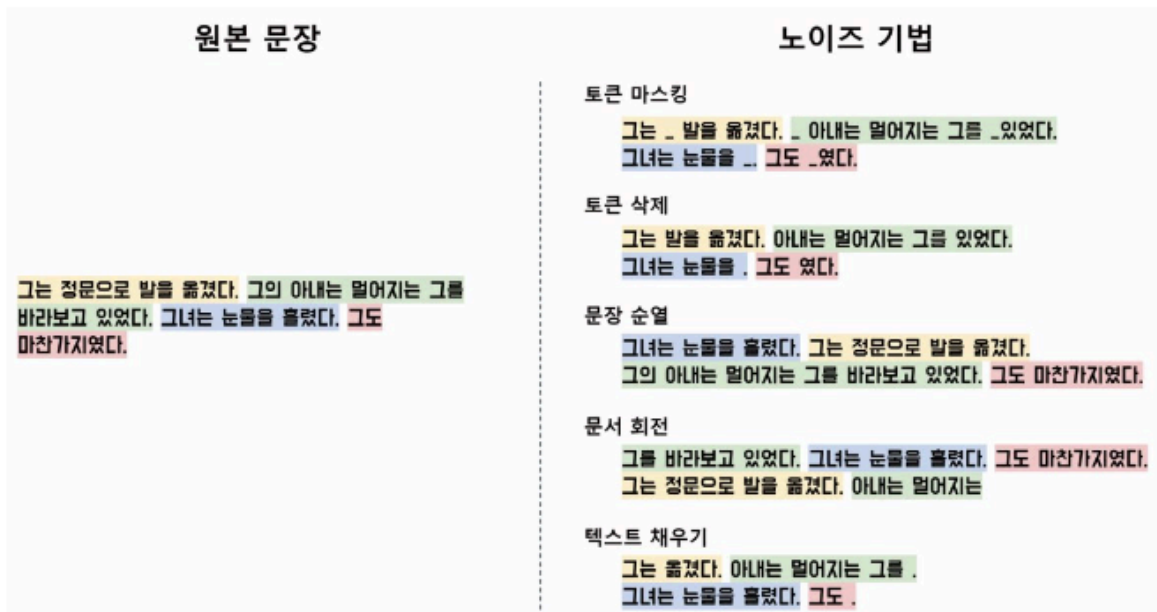


그림 7.17 BART 노이즈 기법

미세 조정 방법 : 미세 조정 시 각 다운스트림 작업에 맞게 입력 문장 구성 필요

문장 분류: 입력 문장을 인코더와 디코더에 동일하게 입력, 디코더의 마지막 은닉 상태를 선형 분류기로 | 입력값으로 사용

토큰 분류: 디코더의 각 시점별 마지막 은닉 상태를 토큰 분류기의 입력값으로 사용. 전체 입력과 디코더의 각 시점별 은닉 상태와의 어텐션 연산을 수행

BART는 트랜스포머 디코더 사용 -> BERT가 해결하지 못한 문장 생성 작업 수행 가능

- 추상적 질의응답: 입력값을 조작해 출력 생성

- 문장 요약: 같은 작업에 적합

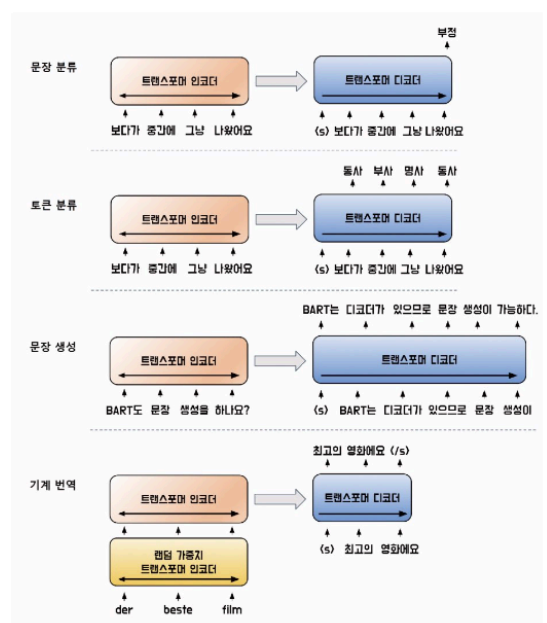


그림 7.18 BART 미세 조정 구조

ELECTRA(Efficiently Learning an Encoder that Classifies Token Replacements Accurately)

입력 마스킹X -> **생성자**와 **판별자**를 사용해 사전 학습 수행

생성 모델: 실제 데이터와 비슷하게 토큰 생성-> 토큰 대체

판별 모델: 생성 모델이 만든 데이터& 실제 데이터 비교, 구분

변환기 모델인 생성자와 판별자 학습: GAN과 유사한 방법으로 학습 수행

- 더 효율적인 학습 가능, 더 빠른 학습
- 더 다양한 문장 생성 가능
- BERT보다 적은 매개변수 -> 모델 크기 감소-> 빠른 속도 및 적은 메모리

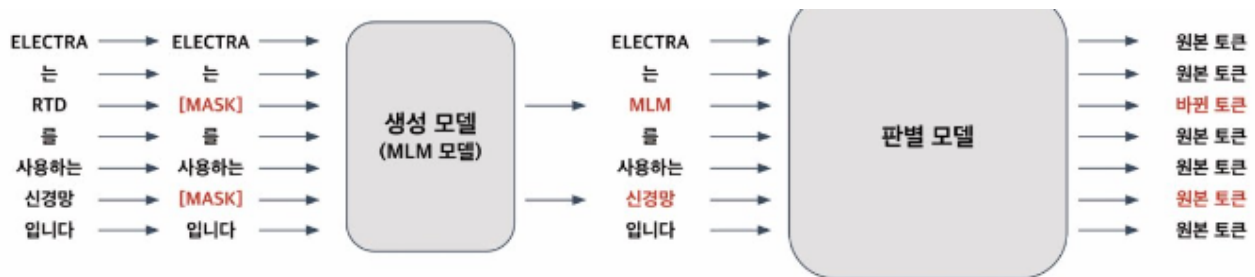
사전 학습 방법:

생성자 모델: 트랜스포머 인코더 구조, 입력 문장의 일부 토큰 마스크 처리 -> 예측을 통한 학습

- BERT의 마스킹된 언어 모델과 동일

판별자 모델: 트랜스포머 인코더 구조, 각 입력 토큰이 원본 문장이 토큰인지, 생성자 모델로 인해 바뀐 토큰인지 맞히며 학습(Replaced Token Detection, RTD)

GAN과 차이점: - 생성자 모델이 토큰을 정확히 예측한 경우 해당 토큰은 생성된 토큰이 아닌 **원본 토큰으로 인식**
- 생성 모델은 마스킹된 언어 모델을 통해 학습, 판별 모델은 각 토큰이 바뀐 토큰인지 원본 토큰이
냐 구분하도록 학습 (GAN처럼 적대적 학습이 아님)
- GAN은 완전한 노이즈 벡터를 입력받아 생성 <-> ELECTRA는 일부 토큰이 마스크 처리된 텍스트를 입력으로 받음



T5(Text-to-Text Transfer Transformer)

인코더-디코더 모델을 바탕으로 GLUE, SuperGLUE, CNN/DailyMail 등에서 높은 성능(State-of-the-Art, SOTA) 달성

자연어 처리 작업에서 높은 성능을 보임

모든 작업을 텍스트 입력과 텍스트 출력으로 변환하는 Text-to-Text 구조 -> 입력과 출력 간의 관계를 더 세밀하게 다룰 수 있음

사전 학습 후 미세 조정 단계에서 해당 작업의 데이터를 이용해 모델 조정->최적의 성능

-CoLA dataset: 문법적으로 허용 가능한 문장 구분

-STS-B dataset: 문장 간 의미적 유사성 측정

사전 학습 방식

비지도 학습 방식, 입력 문장의 일부 구간 마스킹-> 입력 시퀀스 처리, 출력 시퀀스는 실제 마스킹된 토큰& 토큰의 연결로 구성

센티널 토큰: 문장마다 유일한 마스크 토큰을 의미. <extra_id_0>~<extra_id_99> 까지 100개 기본값 사용

Ex) '인코더-디코더 모델 구조'; 마스킹: 인코더, 디코더

-> 입력 토큰의 센티널 토큰: [<extra_id_0> - <extra_id_1>, 모델 구조]; 출력 토큰: [인코더, <extra_id_0>, 디코더, <extra_id_1>]

사전 학습 완료 후 미세 조정을 통해 다양한 자연어 처리 작업 수행

T5의 미세 조정: 지도 학습 방식으로 학습.

- 번역, 언어 수용성, 문장 유사도, 문서 요약 등 작업에 활용

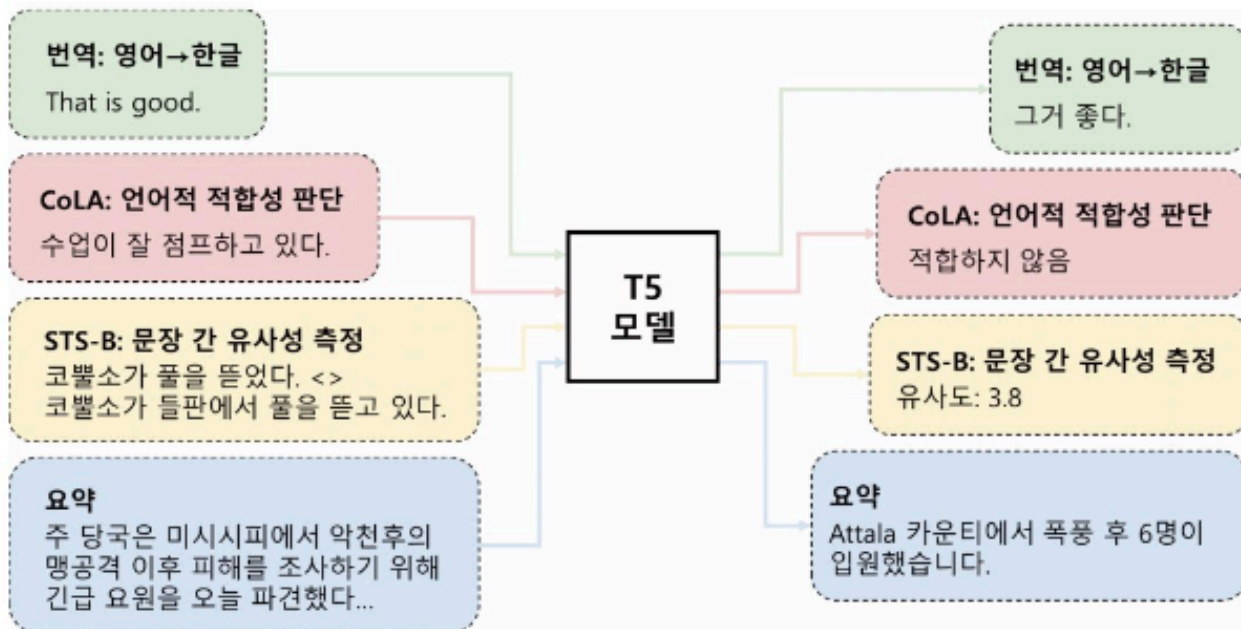


그림 7.21 T5 모델 학습 유형

문장이 인코딩 되기 전에 “translate to English from German”, “summarize:” 같은 작업 토큰 문장 앞에 추가할 경우 작업 토큰도 함께 학습-> 자연어 처리 작업에서 높은 성능 발휘 가능

- T5가 BERT, BART, ELECTRA를 대체하지 않고 각각 모델이 아직 사용되는 이유?
- ELECTRA는 GAN과 유사한 생성자-판별자 구조를 사용하지만 적대적 학습 대신 RTD 방식을 사용한다. ELECTRA가 이러한 구조를 채택한 이유는 무엇이며, 이러한 방식의 장점이 있다면 왜 GAN 기반 생성 모델들도 동일한 방식을 사용하지 않는가?
- T5에서 사전 학습은 비지도, 미세 조정은 지도 학습 방식인 이유가 따로 있는지?